МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8
«Компьютерные науки и прикладная математика»
Кафедра 806

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

по учебной дисциплине «Операционные системы и архитектура компьютера»
на тему «Решение системы линейных уравнений методом Гаусса»
Вариант 10

Выполнил:
Юрин Матвей Юрьевич
М8О-206БВ-25
Проверил:
Ефимов Д.А.

Москва 2026

СОДЕРЖАНИЕ

1 Цель работы и задание 3

2 Сборка и запуск 3

3 Как работает метод Гаусса 3

4 Работа с потоками 4

5 Проверка программы 4

6 Просмотр потоков 5

7 Измерение времени 5

8 Графики и разбор результатов 6

9 Исходный код 8

10 Вывод 12

1 Цель работы и задание

Цель работы — научиться создавать потоки и согласовывать их работу. Для варианта 10 нужно решить систему линейных уравнений методом Гаусса. Количество потоков задаётся при запуске. Также нужно показать потоки средствами ОС и сравнить время работы при разных размерах матрицы.

Программа написана на C11. Для потоков используется pthread, для сборки — CMake. Все вычисления находятся в одном файле main.c.

2 Сборка и запуск

Команды выполняются из корня проекта:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_C_FLAGS_RELEASE='-O2 -DNDEBUG'
cmake --build build
./build/OS_Lab3_Variant10 --size 1000 --threads 4
./build/OS_Lab3_Variant10 --file tests/three.txt --threads 1
```

Ключ size создаёт систему заданного размера, file читает её из файла. Одновременно указывать оба ключа нельзя. Ключ threads задаёт общее число потоков, включая основной. При значении 1 программа работает последовательно. В CLion достаточно открыть корневую папку и указать эти же аргументы запуска.

В файле сначала записывается число неизвестных, затем коэффициенты каждого уравнения и его правая часть. Например:

```
3
2 1 -1 8
-3 -1 2 -11
-2 1 2 -3
```

У этой системы ответ $x = (2, 3, -1)$.

3 Как работает метод Гаусса

Система записывается в виде $Ax = b$. Матрица A содержит коэффициенты, вектор b — правые части, а x — неизвестные. В программе коэффициенты и правые части хранятся вместе, в расширенной матрице из n строк и n+1 столбцов. Память выделяется через malloc, числа имеют тип double.

На шаге k выбирается наибольший по модулю элемент в текущем столбце среди строк от k до n-1. Его строка меняется местами с ведущей. Это позволяет не делить на ноль и уменьшает ошибки округления. Если подходящего ведущего элемента нет, программа сообщает, что безопасное единственное решение получить нельзя.

Дальше из каждой нижней строки вычитается ведущая строка, умноженная на коэффициент a_{ik}/a_{kk}. Так под диагональю появляются нули. Это прямой ход. После него матрица становится верхнетреугольной.

Обратный ход начинается с последнего уравнения. Сначала находится последняя неизвестная, затем предыдущая и так далее. Уже найденные значения подставляются в более ранние уравнения.

4 Работа с потоками

Параллельно можно изменять разные строки ниже ведущей. На одном шаге все потоки читают одну ведущую строку, но каждый записывает результат только в свой диапазон. Поэтому они не меняют одни и те же элементы.

Рабочие потоки создаются один раз. Основной поток тоже обрабатывает строки. Например, при threads=4 создаются три рабочих потока, а четвёртым участником является main. Если строк меньше заданного лимита, число участников ограничивается размером матрицы.

Новый шаг нельзя начинать, пока не закончен предыдущий: следующая ведущая строка ещё изменяется. Поэтому основной поток публикует номер шага и будит рабочих. После обработки строк каждый рабочий увеличивает счётчик done. Main ждёт всех рабочих и только после этого выбирает следующую опору. Номер generation позволяет отличить новое задание от уже выполненного.

Общие счётчики защищены mutex. Условные переменные work и finished нужны, чтобы потоки спали во время ожидания. Проверка условия стоит в цикле while, поскольку пробуждение само по себе ещё не означает, что нужное условие выполнено. Обратный ход оставлен последовательным.

Функция pthread	Что она делает в этой работе
pthread_create	Запускает рабочий поток.
pthread_join	Ждёт завершения рабочего перед освобождением памяти.
pthread_mutex_lock/unlock	Защищает состояние текущего шага.
pthread_cond_wait	Отпускает mutex и засыпает; перед возвратом снова захватывает mutex.
pthread_cond_broadcast	Будит всех рабочих при новом задании или завершении.
pthread_cond_signal	Последний рабочий сообщает main, что шаг закончен.

5 Проверка программы

После решения программа подставляет x в исходную систему и считает невязку $r = Ax - b$. Для созданных систем она ещё сравнивает x с заранее известным ответом. Матрицы генератора имеют диагональное преобладание, поэтому случайная вырожденная система не мешает измерениям.

Для повторения основных проверок оставлен небольшой сценарий:

```
python3 tests/run_tests.py
ctest --test-dir build --output-on-failure
```

Каждый случай проверен с одним и четырьмя потоками.

Проверка	Результат
Система 2x2	Ответ (2,1), проверка пройдена.
Система 3x3	Ответ (2,3,-1), проверка пройдена.
Перестановка строк	Ответ (1,2,-1), проверка пройдена.
Вырожденная система	Программа завершилась с сообщением об ошибке.
Генерация n=100 и n=1000	Проверка невязки и известного ответа пройдена.

Всего сценарий выполняет 12 проверок. Все они прошли. Значения ответа могут отличаться в последних знаках double, поэтому сравнение делается с учётом погрешности.

6 Просмотр потоков

В macOS потоки процесса можно посмотреть через ps:

```
./build/OS_Lab3_Variant10 --size 1000 --threads 4 --demo 20 &
pid=$!
ps -M -p "$pid"
wait "$pid"
```

Ключ demo добавляет паузу после создания рабочих. За это время удобно выполнить ps. После паузы тот же набор потоков решает систему. Ниже приведён настоящий вывод запуска с четырьмя потоками. Для читаемости из первой строки убран длинный путь программы:

```
$ ps -M -p 13492
USER         PID   TT   %CPU STAT PRI     STIME     UTIME
matveyyuri 13492   ??    0.0 S    31T   0:00.00   0:00.00
           13492         0.0 S    31T   0:00.00   0:00.00
           13492         0.0 S    31T   0:00.00   0:00.00
           13492         0.0 S    31T   0:00.00   0:00.00
```

После заголовка видно четыре строки, то есть четыре потока вместе с main. Полные результаты проверок лимитов 1, 2, 4 и 8 сохранены в results/threads.txt. В Ubuntu аналогичная команда — ps -T -p PID; в этой работе она не проверялась.

7 Измерение времени

Замеры сделаны на MacBook с Apple M1 и 8 ГиБ памяти, macOS 26.5.1. Использовались Apple Clang 21.0.0 и Release-сборка с оптимизацией -O2. Для каждого размера исходные данные одинаковы при любом числе потоков.

Таймер CLOCK_MONOTONIC охватывает прямой ход, ожидание рабочих и обратный ход. Генерация, создание и завершение потоков, проверка ответа и печать в это время не входят. Для каждой пары размера и числа потоков выполнен один прогрев и семь замеров. В таблице указана медиана этих семи значений. Всего сохранено 112 измерений.

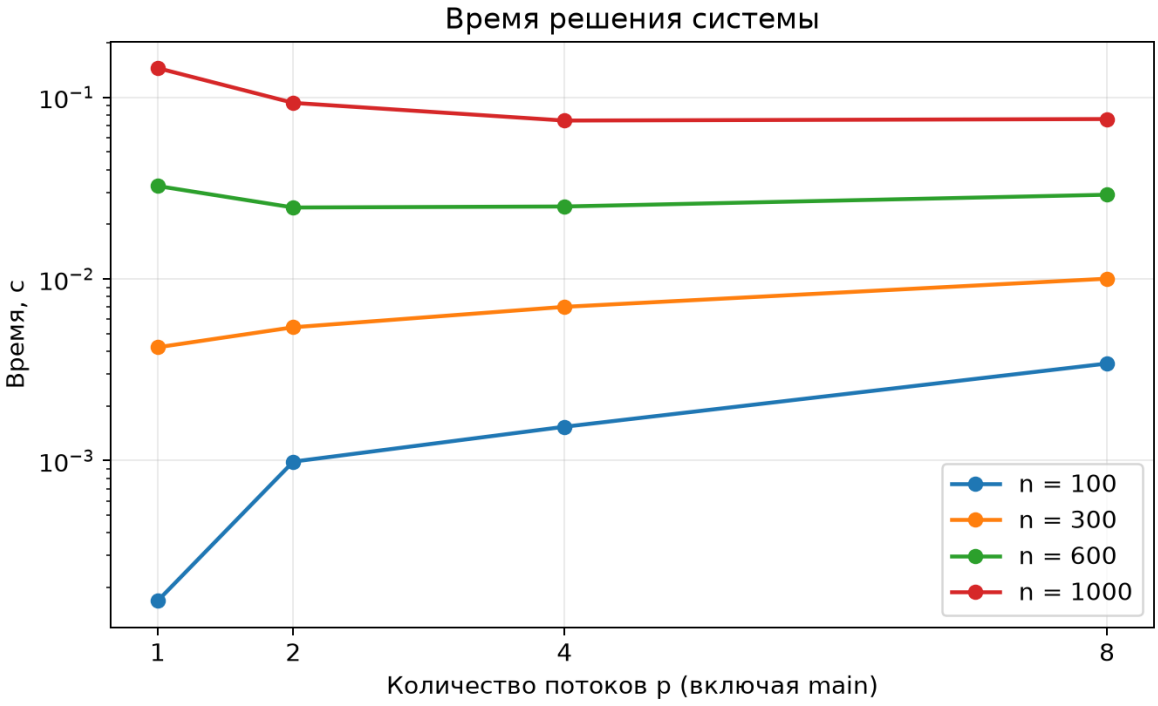
Ускорение показывает, во сколько раз уменьшилось время, а эффективность — какая доля от числа участников получилась в ускорении:

$$S(p) = \frac{T(1)}{T(p)}, \qquad E(p) = \frac{S(p)}{p}.$$

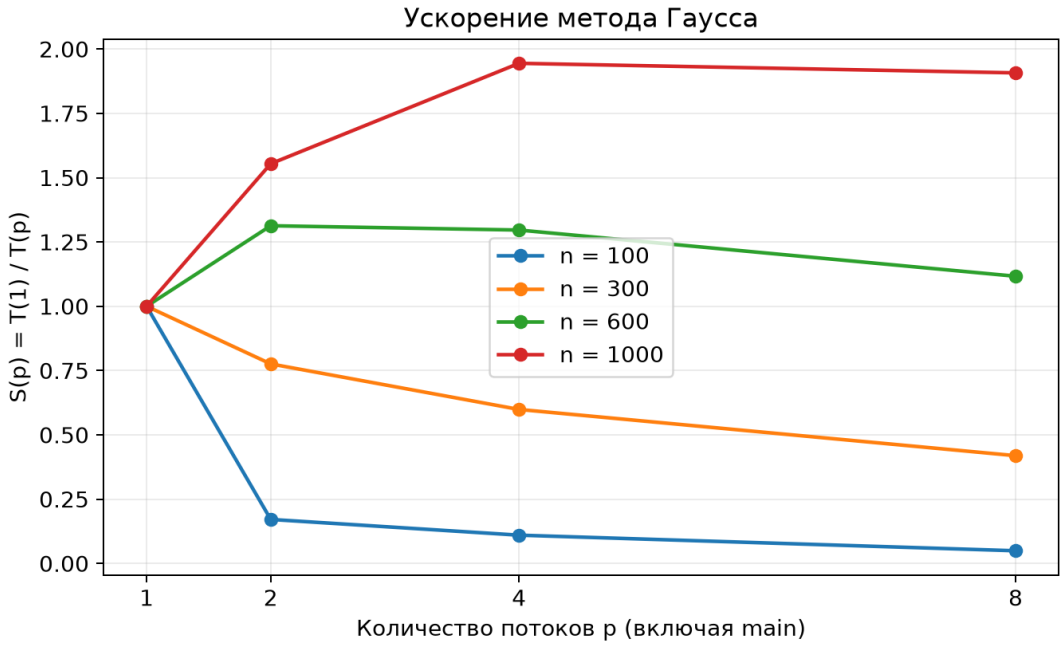
Здесь p включает основной поток. В конце прямого хода нижних строк становится меньше, поэтому часть участников может получать пустой диапазон.

n	p	Время, мс	Ускорение	Эффективность, %
100	1	0.169	1.000	100.00
100	2	0.987	0.171	8.56
100	4	1.536	0.110	2.75
100	8	3.423	0.049	0.62
300	1	4.206	1.000	100.00
300	2	5.421	0.776	38.79
300	4	7.024	0.599	14.97
300	8	10.037	0.419	5.24
600	1	32.494	1.000	100.00
600	2	24.746	1.313	65.66
600	4	25.066	1.296	32.41
600	8	29.097	1.117	13.96
1000	1	144.897	1.000	100.00
1000	2	93.231	1.554	77.71
1000	4	74.512	1.945	48.62
1000	8	75.963	1.907	23.84

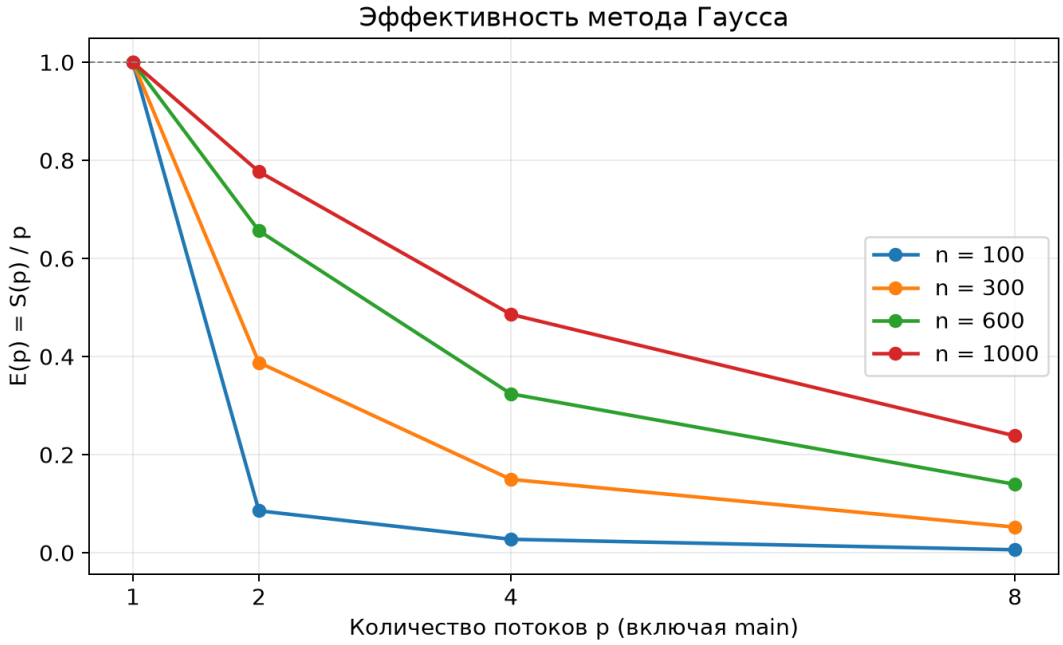
8 Графики и разбор результатов



Время решения. Ось времени логарифмическая.



Ускорение относительно одного потока для каждого размера.



Эффективность использования p участников.

При n=100 время выросло с 0,169 мс на одном потоке до 1,536 мс на четырёх. Работы мало, и ожидания занимают больше времени, чем экономят вычисления. Для n=300 многопоточность тоже оказалась медленнее.

При n=1000 четыре потока уменьшили время со 144,897 до 74,512 мс: ускорение около 1,94, эффективность около 49%. Восемь потоков дополнительного выигрыша не дали; их результаты близки к четырём потокам и имеют разброс.

Линейного ускорения нет: выбор опоры и обратный ход последовательны, каждый шаг требует ожидания, а число оставшихся строк уменьшается. На время также влияют планировщик и другие открытые программы.

9 Исходный код

Программа находится в main.c. Ниже приведён её полный исходный код.

```
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <float.h>
#include <math.h>
#include <pthread.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>

typedef struct {
    double *a;
    int n, threads, k, generation, done, stop;
    pthread_mutex_t mutex;
    pthread_cond_t work, finished;
} Pool;

typedef struct {
    Pool *pool;
    int id;
} Worker;

static void check_pthread(int error, const char *name) {
    if (error != 0) {
        fprintf(stderr, "%s: %s\n", name, strerror(error));
        exit(EXIT_FAILURE);
    }
}

static void help(void) {
    puts("Usage: OS_Lab3_Variant10 (--size N | --file PATH) --threads P [--demo SEC]\n"
         "  --size N     Generate a reproducible system, 1 <= N <= 10000\n"
         "  --file PATH  Read N, then N rows of N coefficients and a right side\n"
         "  --threads P  Total thread limit including main, 1 <= P <= 256\n"
         "  --demo SEC   Pause with workers alive before timing, 1..300 seconds\n"
         "  --help       Show this help\n"
         "Actual threads = min(P, N). Times exclude input, pool setup and checking.");
}

static int number(const char *s, int limit) {
    char *end;
    errno = 0;
    long value = strtol(s, &end, 10);
    if (errno || end == s || *end || value < 1 || value > limit) return 0;
    return (int)value;
}

static double seconds(void) {
    struct timespec t;
    if (clock_gettime(CLOCK_MONOTONIC, &t) != 0) {
        perror("clock_gettime");
        exit(EXIT_FAILURE);
    }
    return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
}

static double known_x(int j) {
    return (double)(j % 7 - 3);
}

static void generate(double *a, int n) {
    uint32_t seed = 10;
    size_t width = (size_t)n + 1;
    for (int i = 0; i < n; ++i) {
        double sum = 0, b = 0;
```

```
        for (int j = 0; j < n; ++j) {
            seed = seed * UINT32_C(1664525) + UINT32_C(1013904223);
            double value = (double)((int)(seed % 21) - 10);
            a[(size_t)i * width + j] = value;
            if (i != j) sum += fabs(value);
        }
        a[(size_t)i * width + i] = sum + 1;
        for (int j = 0; j < n; ++j)
            b += a[(size_t)i * width + j] * known_x(j);
        a[(size_t)i * width + n] = b;
    }
}

/* Each participant changes a disjoint, contiguous range below row k. */
static void eliminate(Pool *pool, int id, int k) {
    int n = pool->n;
    size_t width = (size_t)n + 1;
    int rows = n - k - 1;
    int start = k + 1 + rows * id / pool->threads;
    int end = k + 1 + rows * (id + 1) / pool->threads;
    const double *pivot = pool->a + (size_t)k * width;
    for (int i = start; i < end; ++i) {
        double *row = pool->a + (size_t)i * width;
        double factor = row[k] / pivot[k];
        row[k] = 0;
        for (int j = k + 1; j <= n; ++j)
            row[j] -= factor * pivot[j];
    }
}

static void *worker_main(void *arg) {
    Worker *worker = arg;
    Pool *pool = worker->pool;
    int seen = 0;
    check_pthread(pthread_mutex_lock(&pool->mutex), "mutex_lock");
    for (;;) {
        while (!pool->stop && pool->generation == seen)
            check_pthread(pthread_cond_wait(&pool->work, &pool->mutex), "cond_wait");
        if (pool->stop) break;
        seen = pool->generation;
        int k = pool->k;
        check_pthread(pthread_mutex_unlock(&pool->mutex), "mutex_unlock");
        eliminate(pool, worker->id, k);
        check_pthread(pthread_mutex_lock(&pool->mutex), "mutex_lock");
        ++pool->done;
        if (pool->done == pool->threads - 1)
            check_pthread(pthread_cond_signal(&pool->finished), "cond_signal");
    }
    check_pthread(pthread_mutex_unlock(&pool->mutex), "mutex_unlock");
    return NULL;
}

static void stop_pool(Pool *pool, pthread_t *ids, int created) {
    check_pthread(pthread_mutex_lock(&pool->mutex), "mutex_lock");
    pool->stop = 1;
    check_pthread(pthread_cond_broadcast(&pool->work), "cond_broadcast");
    check_pthread(pthread_mutex_unlock(&pool->mutex), "mutex_unlock");
    for (int i = 0; i < created; ++i)
        check_pthread(pthread_join(ids[i], NULL), "pthread_join");
    check_pthread(pthread_cond_destroy(&pool->finished), "cond_destroy");
    check_pthread(pthread_cond_destroy(&pool->work), "cond_destroy");
    check_pthread(pthread_mutex_destroy(&pool->mutex), "mutex_destroy");
}

static int start_pool(Pool *pool, pthread_t *ids, Worker *workers) {
    check_pthread(pthread_mutex_init(&pool->mutex, NULL), "mutex_init");
    check_pthread(pthread_cond_init(&pool->work, NULL), "cond_init");
    check_pthread(pthread_cond_init(&pool->finished, NULL), "cond_init");
    for (int i = 0; i < pool->threads - 1; ++i) {
        workers[i].pool = pool;
        workers[i].id = i + 1;
```

```
        int error = pthread_create(&ids[i], NULL, worker_main, &workers[i]);
        if (error != 0) {
            fprintf(stderr, "pthread_create: %s\n", strerror(error));
            stop_pool(pool, ids, i);
            return 0;
        }
    }
    return 1;
}

static int gauss(Pool *pool, double *x) {
    int n = pool->n;
    double *a = pool->a;
    size_t width = (size_t)n + 1;
    double scale = 0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            scale = fmax(scale, fabs(a[(size_t)i * width + j]));
    double tolerance = DBL_EPSILON * n * scale;
    for (int k = 0; k < n; ++k) {
        int best = k;
        for (int i = k + 1; i < n; ++i)
            if (fabs(a[(size_t)i * width + k]) > fabs(a[(size_t)best * width + k]))
                best = i;
        double pivot = fabs(a[(size_t)best * width + k]);
        if (!isfinite(pivot) || pivot <= tolerance) return 0;
        if (best != k) {
            for (int j = 0; j <= n; ++j) {
                double tmp = a[(size_t)k * width + j];
                a[(size_t)k * width + j] = a[(size_t)best * width + j];
                a[(size_t)best * width + j] = tmp;
            }
        }
        if (k == n - 1) break;
        if (pool->threads == 1) {
            eliminate(pool, 0, k);
        } else {
            check_pthread(pthread_mutex_lock(&pool->mutex), "mutex_lock");
            pool->k = k;
            pool->done = 0;
            ++pool->generation;
            check_pthread(pthread_cond_broadcast(&pool->work), "cond_broadcast");
            check_pthread(pthread_mutex_unlock(&pool->mutex), "mutex_unlock");
            eliminate(pool, 0, k);
            check_pthread(pthread_mutex_lock(&pool->mutex), "mutex_lock");
            while (pool->done != pool->threads - 1)
                check_pthread(pthread_cond_wait(&pool->finished, &pool->mutex), "cond_wait");
            check_pthread(pthread_mutex_unlock(&pool->mutex), "mutex_unlock");
        }
    }
    for (int i = n - 1; i >= 0; --i) {
        double sum = a[(size_t)i * width + n];
        for (int j = i + 1; j < n; ++j) sum -= a[(size_t)i * width + j] * x[j];
        x[i] = sum / a[(size_t)i * width + i];
        if (!isfinite(x[i])) return 0;
    }
    return 1;
}

static int verify(const double *a, const double *x, int n, int generated) {
    long double residual = 0, norm_a = 0, norm_b = 0, norm_x = 0, error = 0;
    size_t width = (size_t)n + 1;
    for (int i = 0; i < n; ++i) {
        long double sum = 0, row_sum = 0;
        for (int j = 0; j < n; ++j) {
            sum += (long double)a[(size_t)i * width + j] * x[j];
            row_sum += fabsl(a[(size_t)i * width + j]);
        }
        if (!isfinite(sum) || !isfinite(row_sum)) {
            puts("verification=FAIL (numeric overflow during checking)");
            return 0;
```

```
        }
        residual = fmaxl(residual, fabsl(sum - a[(size_t)i * width + n]));
        norm_a = fmaxl(norm_a, row_sum);
        norm_b = fmaxl(norm_b, fabsl(a[(size_t)i * width + n]));
        norm_x = fmaxl(norm_x, fabsl(x[i]));
        if (generated) error = fmaxl(error, fabsl(x[i] - known_x(i)));
    }
    long double denominator = norm_a * norm_x + norm_b;
    long double relative = denominator > 0 ? residual / denominator : residual;
    int ok = isfinite(denominator) && isfinite(relative) && relative <= 1e-10L;
    if (generated && error > 1e-8L) ok = 0;
    printf("residual_inf=%.12Le\nrelative_residual=%.12Le\n", residual, relative);
    if (generated) printf("max_solution_error=%.12Le\n", error);
    printf("verification=%s\n", ok ? "PASS" : "FAIL");
    return ok;
}

int main(int argc, char **argv) {
    int n = 0, threads = 0, demo = 0;
    const char *file = NULL;
    for (int i = 1; i < argc; ++i) {
        if (strcmp(argv[i], "--help") == 0 && argc == 2) { help(); return 0; }
        if (i + 1 == argc) goto bad_args;
        const char *option = argv[i++];
        if (strcmp(option, "--size") == 0 && !n && !file) {
            n = number(argv[i], 10000);
            if (!n) goto bad_args;
        } else if (strcmp(option, "--file") == 0 && !file && !n) {
            file = argv[i];
        } else if (strcmp(option, "--threads") == 0 && !threads) {
            threads = number(argv[i], 256);
            if (!threads) goto bad_args;
        } else if (strcmp(option, "--demo") == 0 && !demo) {
            demo = number(argv[i], 300);
            if (!demo) goto bad_args;
        } else goto bad_args;
    }
    if ((!n && !file) || !threads) goto bad_args;
    FILE *input = NULL;
    if (file) {
        input = fopen(file, "r");
        if (!input) { perror(file); return 1; }
        char token[128];
        if (fscanf(input, "%127s", token) != 1 || !(n = number(token, 10000))) {
            fprintf(stderr, "Invalid matrix size in file.\n");
            fclose(input);
            return 1;
        }
    }
    size_t count = (size_t)n * ((size_t)n + 1);
    double *a = malloc(count * sizeof(*a));
    double *original = malloc(count * sizeof(*original));
    double *x = malloc((size_t)n * sizeof(*x));
    int actual = threads < n ? threads : n;
    pthread_t *ids = calloc((size_t)actual, sizeof(*ids));
    Worker *workers = calloc((size_t)actual, sizeof(*workers));
    int result = 1;
    if (!a || !original || !x || !ids || !workers) {
        fprintf(stderr, "Not enough memory.\n");
        if (input) fclose(input);
        goto cleanup;
    }
    if (input) {
        for (size_t i = 0; i < count; ++i) {
            if (fscanf(input, "%lf", &a[i]) != 1 || !isfinite(a[i])) {
                fprintf(stderr, "Expected finite matrix coefficients and right sides.\n");
                fclose(input);
                goto cleanup;
            }
        }
        char extra[2];
```

```
        int extra_result = fscanf(input, "%1s", extra);
        int read_error = ferror(input);
        fclose(input);
        if (extra_result == 1 || read_error) {
            fprintf(stderr, "Extra data or read error in matrix file.\n");
            goto cleanup;
        }
    } else generate(a, n);
    memcpy(original, a, count * sizeof(*a));
    Pool pool = {.a = a, .n = n, .threads = actual};
    if (!start_pool(&pool, ids, workers)) goto cleanup;
    printf("size=%d\nthread_limit=%d\nactual_threads=%d\n", n, threads, actual);
    if (demo) {
        printf("demo_pid=%ld\ndemo_seconds=%d\n", (long)getpid(), demo);
        fflush(stdout);
        struct timespec delay = {.tv_sec = demo, .tv_nsec = 0};
        while (nanosleep(&delay, &delay) == -1 && errno == EINTR) {}
    }
    double start = seconds();
    int solved = gauss(&pool, x);
    double elapsed = seconds() - start;
    stop_pool(&pool, ids, actual - 1);
    printf("time_seconds=%.9f\n", elapsed);
    if (!solved) {
        fprintf(stderr, "Singular or numerically unsafe system: unique solution unavailable.\n");
        result = 2;
        goto cleanup;
    }
    result = verify(original, x, n, file == NULL) ? 0 : 3;
    if (n <= 10)
        for (int i = 0; i < n; ++i) printf("x[%d]=%.17g\n", i + 1, x[i]);
cleanup:
    free(workers);
    free(ids);
    free(x);
    free(original);
    free(a);
    return result;
bad_args:
    fprintf(stderr, "Invalid arguments. Use --help.\n");
    return 1;
}
```

10 Вывод

В работе реализовано решение системы методом Гаусса с помощью pthread. Потоки обрабатывают разные строки и согласуют переход к следующему шагу через mutex и условные переменные. Основные проверки прошли, количество потоков удалось показать через ps.

Замеры показали, что для маленьких матриц проще и быстрее использовать один поток. Для матрицы 1000x1000 четыре потока дали ускорение около 1,94. Увеличение числа потоков само по себе не гарантирует уменьшения времени.